DEPARTMENT OF COMPUTER & INFORMATION SYSTEMS ENGINEERING
BACHELORS IN COMPUTER SYSTEMS ENGINEERING
Course Code: CS-324
Course Title: Machine Learning
Complex Engineering Problem
TE Batch 2023, Spring Semester 2026
Grading Rubric 1: Final Model Implementation and Evaluation (Milestone 3)
TERM PROJECT

Group Members:

Student No.	Name	Roll No.
S1	Sitwat Samara	CS-23105
S2	Manahil Adeel	CS-23106
S3	Talha Adil	CS-23114
S4	MariumShad	CS-23116

CRITERIA AND SCALES				**Marks Obtained**			
				S1	**S2**	**S3**	**S4**
Criterion 1: Model implementation and fulfillment of technical requirements. *(CPA-1, CPA-3)* **[3 marks]**							
1	2	3	4				
Required models are not implemented or are incorrectly implemented.	Some required models are implemented, but with major issues.	All required models are implemented correctly with minor issues.	All required models are correctly implemented and fully meet the project requirements.				
Criterion 2: Comparative analysis, evaluation, and discussion. *(CPA-1, CPA-2)* **[2 marks]**							
1	2	3	4				
No meaningful evaluation or comparison is provided.	Limited evaluation and comparison with weak analysis.	Adequate evaluation and comparison with reasonable analysis.	Thorough evaluation, clear comparisons, and insightful discussion of results.				
Criterion 3: Report quality and adherence to format. **[2 marks]**							
1	2	3	4				
The report is poorly written and does not follow the given format.	The report partially follows the format and lacks clarity.	The report follows the format with minor formatting or clarity issues.	The report is clear, well written, and fully adheres to the given format and requirements.				
Criterion 4: jIndividual contribution, teamwork, and viva performance. *(CPA-1, CPA-2, CPA-3)* **[3 marks]**							
1	2	3	4				
The student did not contribute meaningfully and could not explain the work.	The student contributed partially and showed limited understanding.	The student contributed satisfactorily and explained the work adequately.	The student demonstrated strong contribution, teamwork, and excellent understanding during viva.				

Final Score = (Criteria_1_score) + (Criteria_2_score) + (Criteria_3_score) + (Criteria_4_score) = _____

_____
Teacher's Signature

1. Data Preprocessing

1.1 Dataset Collection

The primary objective of this project was to build a system capable of accurately recognizing all 26 alphabets of American Sign Language (ASL). Rather than relying entirely on a publicly available dataset, the team chose to collect a custom image dataset in order to maintain greater control over data quality and to ensure that the images would closely reflect real-world usage conditions. The collection effort was distributed equally among four team members, with each member responsible for capturing approximately 50 images per alphabet letter. This resulted in an initial collection of over 200 images per class. Images were captured using mobile phones and included hand gestures from multiple individuals including team members, friends, and siblings to introduce meaningful diversity in hand shapes, skin tones, and subtle gesture variations. All images were consolidated into a shared Google Drive folder, organized into 26 class-specific subdirectories.

1.2 Dataset Version Decision: Cropped vs. Background Images

During early experiments, the team tested two versions of the dataset: one consisting of full background images and one with cropped hand regions only. The full-background version introduced significant noise into the training process. Since the overall dataset size was relatively small, CNN models trained on background images tended to overfit to background cues rather than learning genuine hand gesture features, leading to poor generalization on unseen inputs. Based on these observations, the team decided to proceed exclusively with the cropped dataset, in which only the hand region is retained. This choice substantially reduced overfitting and improved the model's ability to generalize across different environments.

1.3 Image Resizing

All images were resized to a uniform resolution to ensure compatibility across model architectures. Two resolutions were prepared: 128×128 pixels for all CNN-from-scratch models, which reduces computational load while retaining sufficient spatial detail for hand gesture recognition; and 300×300 pixels for the EfficientNetB3 transfer learning model, which expects higher-resolution inputs as per its pretrained configuration.

1.4 Dataset Cleaning

After resizing, all images were manually reviewed by team members. Images that were blurry, poorly lit, had the hand partially out of frame, or contained ambiguous gestures were removed from the dataset. This cleaning step was critical to preventing models from learning incorrect or misleading patterns from low-quality samples, which would have artificially inflated training accuracy while degrading real-world performance.

1.5 Color Retention and Pixel Normalization

Images were retained in their original RGB color format rather than converting to grayscale. Color information provides useful contextual cues for distinguishing hand regions from backgrounds, as skin tone contrast with surrounding surfaces aids feature extraction. Pixel values, originally in the range [0, 255], were normalized to [0, 1] by dividing by 255. This normalization ensures all input features operate on a consistent numerical scale, stabilizing gradient updates and accelerating convergence during training.

1.6 Data Augmentation

To compensate for the limited dataset size and improve generalization, data augmentation was applied to each original image, generating five additional variants per sample:

- **Horizontal flip:** enables recognition of gestures from both left and right hands.
- **Reduced brightness:** simulates low-light or indoor lighting conditions.
- **Increased brightness:** simulates well-lit or outdoor environments.
- **Slight upward rotation:** introduces pose variation.
- **Slight downward rotation:** further diversifies gesture orientation.

The original image was also retained, bringing each sample to six total versions and resulting in an approximately 5× increase in dataset size overall.

1.7 Dataset Statistics

Property	Value	Notes
Number of Classes	26	One per ASL letter A–Z
Initial Images per Class	~200+	Manually collected by team
Final Images per Class	~1,200	After 5× augmentation
Total Final Dataset Size	~31,200	26 classes × 1,200 images
Input Resolution (CNN Scratch)	128 × 128 px	RGB color
Input Resolution (EfficientNet)	300 × 300 px	RGB color

2. Model Architectures and Machine Learning Algorithm:

Three distinct model families were developed and evaluated: (1) a custom CNN trained from scratch across three sub-configurations, (2) a transfer learning model based on EfficientNetB3 across three sub-configurations, and (3) a classical machine learning pipeline using Random Forest on MediaPipe extracted hand joint features. Each family is described in detail below.

2.1 Model Family 1: Custom CNN Trained from Scratch

A deep convolutional neural network was designed and trained from scratch, incorporating architectural elements from ResNet and MobileNet combined with modern regularization techniques. The architecture is detailed below.

- **Input shape:** (128, 128, 3) — RGB color images.
- **Block 1 (Initial):** 32 filters — extracts low-level features such as edges and textures, preserving spatial resolution through appropriate padding.
- **Blocks 2–5 (Progressive):** Filter counts increase progressively 32→64→128→256→512 . Deeper layers capture increasingly abstract representations such as finger shapes and sign motifs. MaxPooling2D follows each block to halve spatial dimensions.
- **Residual Connections:** Skip connections after every convolutional block, with 1×1 convolutions to match dimensions where needed. This mitigates vanishing gradients and allows identity mappings when additional depth provides no benefit.
- **Depthwise Separable Convolutions (Blocks 4–5):** Standard convolutions replaced with depthwise + pointwise 1×1 operations, drastically reducing parameter count while retaining strong feature extraction.
- **Batch Normalization:** Applied after every convolution and dense layer to normalize activations, stabilize training, and accelerate convergence.
- **SpatialDropout2D:** Drops entire feature maps (channels) rather than individual neurons, discouraging over-reliance on any single feature and improving generalization.
- **L2 Regularization:** Applied to all Dense layers to penalize large weights and reduce overfitting.
- **Classification Head:** GlobalAveragePooling2D → Dense(512) → Dense(256) → Dense(26, softmax). GlobalAveragePooling summarizes each feature map spatially, significantly reducing parameter count compared to Flatten.
- **Loss Function:** CategoricalCrossentropy with label smoothing softens one-hot targets to reduce overconfidence and improve robustness on noisy labels.
- **Optimizer:** AdamW (Adam with decoupled weight decay) adaptive learning rates combined with robust L2 regularization.

- **Callbacks:** ModelCheckpoint, EarlyStopping, ReduceLROnPlateau for training efficiency and recoverability.

Sub-Model Configurations CNN from Scratch

Hyperparameter	CNN Model 1 (Baseline)	CNN Model 2 (Lighter)	CNN Model 3 (Fine-tuned)
Filter Progression	32→64→128→256→512	16→32→64→128→256	16→32→64→128→256
Batch Size	32	32	16
Learning Rate	1e-3	1e-3	1 e -4
Optimizer	Adam	AdamW	AdamW
Label Smoothing	No	Yes	Yes
Dropout Strategy	Uniform	Graduated (depth-increasing	) Graduated (depth-increasing)
Epochs	Standard	Standard	Extended (compensates low LR)

CNN Model 2 introduced two key improvements over the baseline: a lighter filter progression (halved at each block) to reduce overfitting risk on a moderate-sized dataset, and a graduated dropout strategy where rates increase with network depth, providing stronger regularization where overfitting risk is highest. CNN Model 3 further refined dynamics by reducing batch size to 16 for more stable gradient estimates, lowering the learning rate to 1e-4 for smoother convergence, and extending training epochs to compensate for the slower learning rate.

2.2 Model Family 2: Transfer Learning with EfficientNetB 3

EfficientNetB3, pretrained on ImageNet, was selected as the backbone for the transfer learning model. MobileNet was initially evaluated but discarded due to high loss and low accuracy on this task. EfficientNetB3's compound scaling, which balances depth, width, and resolution simultaneously, made it a more suitable choice for this classification task.

- **Backbone:** EfficientNetB3 with include_top=False pretrained ImageNet weights loaded, original classifier removed.
- **Input resolution:** (300, 300, 3) matching EfficientNetB3's expected input dimensions.
- **Custom Classification Head:** GlobalAveragePooling2D → Dense(512, Swish, BatchNorm, Dropout 0.4, L2) → Dense(256, Swish, BatchNorm, Dropout 0.3, L2) → Dense(26, softmax). Swish activation was used for smoother gradient flow compared to ReLU.

- **Phase 1 (Frozen Backbone):** trainable=False on backbone. Only the custom head is trained. BatchNorm layers run in inference mode to use stored moving averages.
- **Phase 2 (Selective Unfreezing):** Upper backbone layers unfrozen from a specified layer onward. Both head and unfrozen backbone trained jointly at a significantly reduced learning rate to adapt features to the ASL domain without catastrophic forgetting.

Sub-Model Configurations EfficientNetB3

Hyperparameter	EfficientNet Model 1	EfficientNet Model 2	EfficientNet Model 3 (Aggressive FT)
Phase 1 Epochs	30	20	20
Phase 2 Epochs	20	20	20
Batch Size	32	16	16
Phase 1 LR	1e-3	5e-4	5 e -4
Phase 2 LR	1e-4	1e-5	2 e -5
Dropout Layer 1 / 2	0.4 / 0.3	0.4 / 0.3	0.45 / 0.35
Fine-tune From Layer	300	300	280 (Block 7 included)
Gradient Clipping	No	No	Yes

EfficientNet Model 3 introduced the most aggressive fine-tuning: a slightly higher Phase 2 learning rate (2e-5) to escape local minima, increased dropout for regularization of unfrozen backbone weights, unfreezing of approximately 20 additional layers (Block 7), and gradient clipping to prevent instability during fine-tuning of deeper backbone layers.

2.3 Model Family 3: Random Forest on MediaPipe Hand Keypoints

A classical machine learning pipeline was implemented using Google MediaPipe to extract 21 hand joint (keypoint) positions, yielding (x, y, z) coordinates for each landmark. This produces a 63- dimensional feature vector per sample, which was stored as a CSV dataset and used to train Random Forest classifiers. For inference, the pipeline first runs MediaPipe on the input image or webcam frame to detect and extract joint positions, then feeds these coordinates directly to the trained classifier for sign prediction.

Sub-Model Configurations — Random Forest

Configuration	RF Model 1 (Grid Search)	RF Model 2 (Random Search)	RF Model 3 (Aggressive)
Search Method	GridSearchCV	RandomizedSearchCV	RandomizedSearchCV
n_estimators	[100, 200, 300]	[100, 150, 200, 250]	randint(150, 450)
max_depth	[None, 10, 20]	[None, 10, 20, 30]	[None, 10, 20, 30]
min_samples_split	[2, 5]	[2, 5, 10]	randint(2, 11)
min_samples_leaf	[1, 2]	[1, 2, 4]	randint(1, 6)
max_features	—	—	[sqrt, log 2]
class_weight	—	—	[None, balanced]
n_iter / CV Folds	Exhaustive / 3	15 / 3	20 / 2

RF Model 1 used exhaustive grid search for solid baseline coverage. RF Model 2 switched to randomized search over a larger parameter space for improved efficiency. RF Model 3 prioritized broader experimentation with wider search ranges, class imbalance handling via class_weight, and feature sampling options.

3. Notable and Unique Aspects of Implementation

- **Custom Hybrid CNN Architecture:** The CNN-from-scratch model blends design principles from ResNet (residual connections), MobileNet (depthwise separable convolutions), and modern training practices (label smoothing, SpatialDropout2D, AdamW). This combination produces an architecture that balances expressive power, computational efficiency, and strong regularization without relying on any pretrained feature representations.
- **Graduated Dropout Strategy:** Rather than applying uniform dropout rates across all layers, dropout was designed to increase progressively with network depth. Shallow layers receive lighter regularization to preserve low-level feature learning, while deeper, more abstract layers receive stronger dropout, directly targeting the layers where overfitting risk is highest.
- **Multi-Phase Transfer Learning with Surgical Fine-Tuning:** EfficientNetB3 was trained using a disciplined two-phase strategy: the backbone was frozen in Phase 1 while only the custom head was trained, then upper backbone layers were selectively unfrozen in Phase 2 at a significantly reduced learning rate, combined with gradient clipping. This mirrors industrial fine-tuning workflows and is rarely implemented with this level of rigor in academic projects.

- **Integration of Deep Learning and Classical ML Pipelines:** The project spans two fundamentally different modeling paradigms: end-to-end deep learning (CNN, EfficientNetB3) operating on raw pixel inputs, and feature-engineered classical ML (Random Forest) operating on geometric keypoint coordinates. This dual-paradigm approach enables a rigorous comparison of representation learning versus handcrafted feature pipelines.
- **MediaPipe-Driven Inference Pipeline:** For all models including random forest inference and real-time webcam testing, a MediaPipe preprocessing stage is used to detect and isolate the hand region before the input is passed to the model. This ensures consistently clean, hand-centered inputs regardless of background conditions, significantly improving robustness across environments.
- **Real-Time Deployment and Live Testing:** All models were evaluated not only on static test sets but also in live webcam conditions, exposing practical deployment challenges such as variable lighting, background clutter, and hand pose variation that static metrics cannot capture.
- **Systematic Hyperparameter Exploration:** Across all three model families, a structured progression of configurations was followed from baseline to progressively refined, with principled justifications for every hyperparameter change. This reflects a scientific experimental workflow rather than ad-hoc tuning.

4. Model Performance Comparison

The table below summarises test-set accuracy and approximate real-world (live/unseen image) accuracy for one representative configuration per model family. Confusion matrices, training/validation accuracy curves, and loss curves for all sub-models are available in the project Google Drive folder and should be reviewed alongside this table.

Model	Test Accuracy	Real-World Accuracy	Overfitting Risk
Custom CNN (from scratch)	~97%+	~90%	Low
EfficientNetB3 (transfer learning)	~95%	~85%	Low to Med
Random Forest (MediaPipe keypoints)	~90%	~75%	Medium

■ *You can view the confusion matrix plots, training curves, and validation curves for each individual model in the **model directory** in our Google Drive, where all outputs and saved files for each model are stored.*

5. Model Performance Analysis

5.1 CNN From Scratch High Accuracy with Strong Generalization

The custom CNN achieved the highest test accuracy among all models at 97%+. However, this figure must be interpreted carefully: the training and test sets share similar characteristics (consistent lighting, controlled backgrounds post-cropping), which can inflate accuracy. The more meaningful indicator is real-world performance, which remained strong at approximately 90%.

This generalization capability is attributable to the model's tailored architecture. With sufficient regularization (SpatialDropout2D, BatchNorm, L2) and training on the augmented cropped dataset, the model learned genuine hand sign features rather than dataset specific shortcuts. Training from scratch on a task-specific dataset allowed the model to develop representations closely aligned with the subtle inter-class differences in ASL gestures for example, similar finger positions for letters such as A, E, and S. The approximately 7 % gap between test and real-world accuracy suggests residual sensitivity to lighting and camera conditions not fully captured by the augmentation pipeline.

5.2 EfficientNetB3 Strong but Slightly Less Specialized

EfficientNetB3 achieved approximately 95% test accuracy, slightly below the custom CNN. In real-world settings, generalization was reasonable but not as robust as the from-scratch model. Two primary factors explain this pattern:

- **Domain Gap:** EfficientNetB3 was pretrained on ImageNet, a dataset of general natural images. Despite fine-tuning, some low-level feature detectors remain partially misaligned with the specific visual characteristics of hand gestures, resulting in slightly weaker discrimination between similar signs.
- **Input Differences:** Differences in scale, color statistics, and noise between training images and live webcam frames can cause degraded performance, particularly when fine-tuned backbone layers retain biases from ImageNet pretraining.

The two-phase training strategy effectively mitigated catastrophic forgetting. The aggressive Model 3 fine tuning which involved unfreezing Block 7 and enabling gradient clipping provided measurable improvement over the baseline EfficientNet configuration, demonstrating the value of careful domain adaptation.

5.3 Random Forest on MediaPipe Features — Limited Real-World Generalization

The Random Forest model achieved approximately 90% test accuracy, competitive with the deep learning models on the controlled dataset. However, real-world accuracy dropped significantly to approximately 75%, revealing a fundamental limitation of the keypoint-based feature representation. Several factors contribute to this performance gap:

- **Feature Incompleteness:** 21 joint positions encode spatial coordinates but omit critical information such as relative finger angles, palm orientation, depth cues, and hand silhouette all of which are important for distinguishing visually similar signs.
- **MediaPipe Extraction Sensitivity:** Keypoint predictions from MediaPipe are sensitive to lighting conditions, occlusion, camera angle, and individual hand morphology. Small extraction errors compound into noisy feature vectors that the classifier cannot reliably distinguish.
- **Limited Representation Power:** The 63-dimensional feature vector, while compact, lacks the representational richness of convolutional feature maps derived from raw pixel data. Signs that differ primarily in subtle finger curvature or orientation may produce nearly identical keypoint coordinates.

Despite these limitations, the Random Forest model is highly computationally efficient and interpretable, making it a viable option for resource-constrained deployments where maximum accuracy is not the primary requirement.

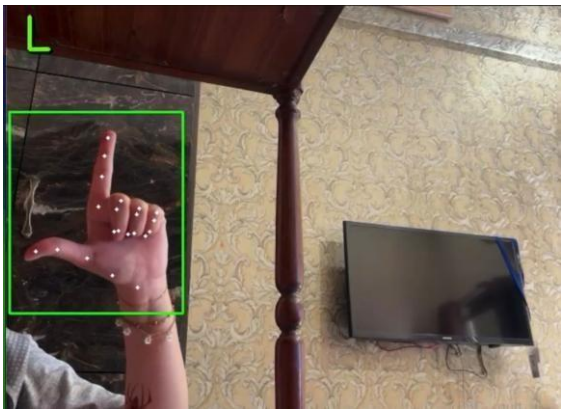
5.4 Effect of Data Split Ratios on Model Performance:

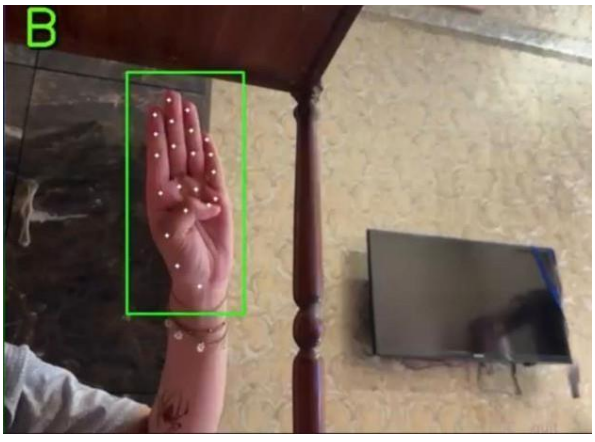
We investigated the effect of changing the data split ratios on the performance of the final selected model by retraining it multiple times using different train–validation–test configurations. In particular, three different splits were evaluated: 70–20–10, 80–10–10, and 60–20–20. For each configuration, the model was trained from scratch to ensure a fair and unbiased comparison of results.

The experiments showed that when the proportion of training data was increased, such as in the 80–10–10 split, the model achieved higher training accuracy but tended to overfit, which resulted in a decrease in test accuracy. In contrast, when the training data was reduced in the 60–20–20 split, the model showed lower overfitting but also struggled to learn effectively due to limited training samples, leading to weaker overall performance. Among all the configurations tested, the 70–20–10 split provided the most balanced results, achieving a good trade-off between learning capability and generalization on unseen data.

6. Test Case Runs

Below are representative test case runs demonstrating the system's classification performance across different input conditions. Each image shows the detected hand gesture with the predicted ASL letter displayed in the top-left corner, along with the MediaPipe keypoint overlay (white dots) used for feature extraction.





All test cases above demonstrate successful real-time detection under varying hand orientations and lighting conditions. The green bounding box indicates the MediaPipe hand detection region, while the white dots represent the 21 extracted keypoint landmarks used as features for the Random Forest model and as the crop region for the CNN-based models.

7. Real-Time Detection Demo

The system supports live, real-time ASL hand sign recognition via webcam input. In real-time mode, each video frame is first processed by the MediaPipe hand detection pipeline to localize and isolate the hand region. The cropped hand region is then passed to the selected model (CNN from scratch, EfficientNetB3, or Random Forest) for classification, and the predicted letter is displayed as an overlay on the live video feed.

A demonstration video showing real-time detection across different hand signs and lighting conditions is available on Google Drive (Test Case Video folder) at the link below:

https://drive.google.com/file/d/1BWibWGO7RvF-XbX7dYOGqEzAKuB350xr/view?usp=drive_link

8. Generative AI Use Declaration

8.1 AI Tools Used

- **GitHub Copilot:** used during code writing sessions for code completion suggestions within notebooks and scripts.
- **ChatGPT / Claude:** used for understanding library specific functions and API usage (TensorFlow, Keras, MediaPipe, scikit learn), particularly in cases where exploring official documentation was time-consuming. Also used to add markdown cells and code comments for notebook readability.

8.2 Specific Purposes of AI Use

- **Library function lookup:** Understanding unfamiliar functions and parameters in TensorFlow, Keras, and MediaPipe rather than reading full documentation pages.
- **Code commenting:** Generating explanatory comments and markdown documentation cells for Jupyter notebooks.
- **Report writing:** Team members independently drafted a 2–3 page summary of the project in plain English. This summary was provided to an AI tool, which reformatted and improved language and structure. All ideas, analysis, decisions, and technical content originated with the team.

8.3 Confirmation

- All project ideas, model design decisions, architectural choices, experimental configurations, and analytical conclusions were conceived and made by the student group.
- All code implementation was done by the group, with AI tools used only for assistance with syntax, library functions, and documentation formatting.
- The final analysis, results interpretation, and all written content reflect our own understanding and work.

AI tools were not used to generate core implementation logic, derive experimental results, or perform any analysis on our behalf.